

Proyecto final de grado



IES Virgen
del Carmen

Jose Buendia Vico

Contenido

1.	Descripción	2
2.	Casos de uso	2
3.	Diagrama de flujo	3
4.	Bocetos	4
5.	Esquema de base de datos	6
5.2.	Modelo entidad relación.....	6
6.	Configuración del entorno	7
6.1.	Preparación de la carpeta y repositorio	7
6.2.	Creación del Backend	7
6.3	Creación del Frontend.....	7
7.	Backend	8
7.1	Creación de la base de datos	8
7.2	Configurar base de datos en el archivo .env	8
7.3	Crear los modelos y migraciones	8
8.	Configurar los migrations	9
9.	Crear las relaciones entre tablas	9
10.	Creamos el controlador	9
11.	Configuración de Rutas (API)	9
12.	Población de la Base de Datos	10
13.	Controladores de Negocio	10
14.	Seguridad y Autenticación (Laravel Sanctum)	11

15.	Configuración de CORS	11
-----	-----------------------------	----

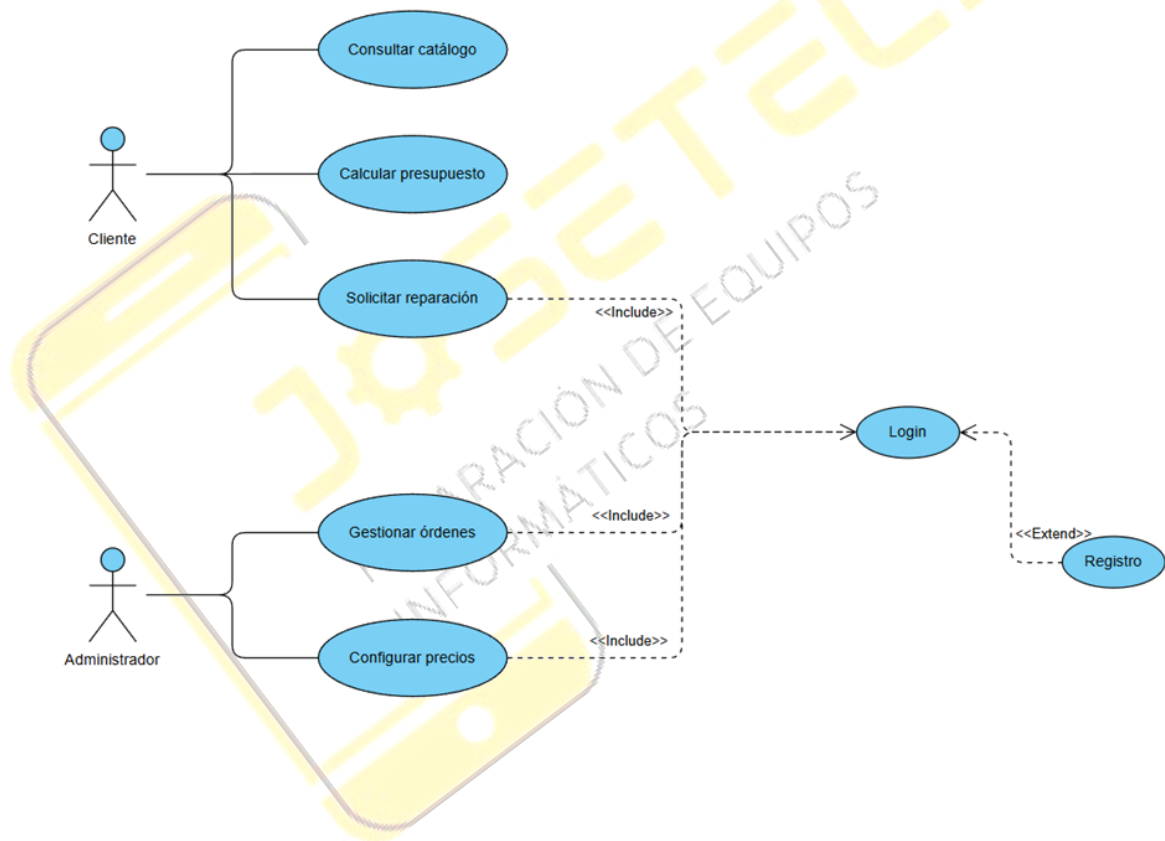


1. Descripción

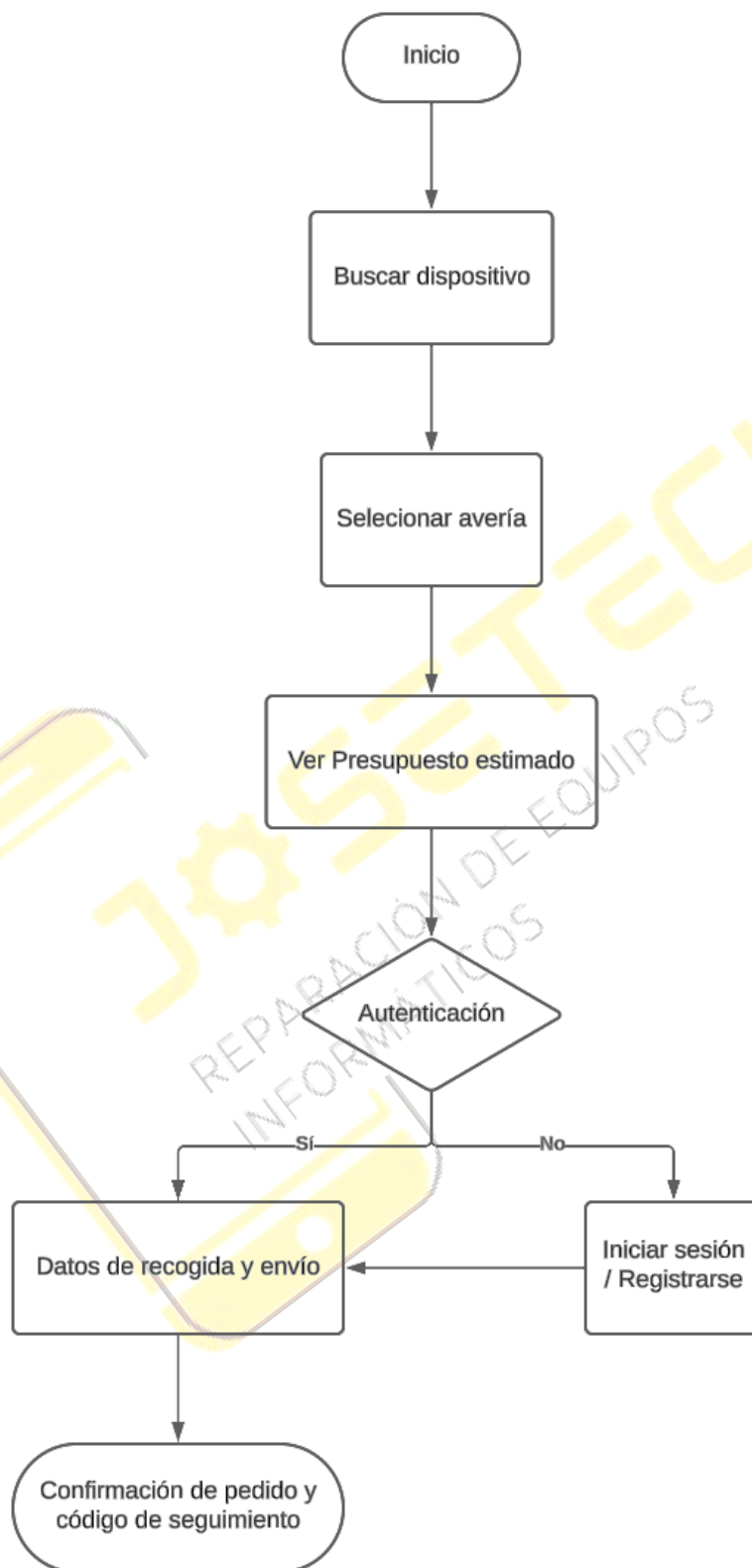
Este proyecto consiste en una página web de servicios de reparaciones de equipos informáticos, donde cada cliente puede seleccionar su modelo de dispositivo (móvil, consola, ordenador, etc.), en esta página podrá consultar el coste de reparación de cada equipo según su componente dañado, también puede solicitar su reparación y consultar la fase de reparación.

Por otro lado, tenemos la parte del administrador que puede gestionar el estado de los pedidos y configurar los precios.

2. Casos de uso



3. Diagrama de flujo



Logo José Tech

Inicio

Presupuesto

Login

¿Se te ha roto tu dispositivo?

Te lo reparamos sin que salgas de casa.

Busca tu modelo (Ej: iPhone 13, PS5...)

O

Icono Móviles

Icono Consolas

Icono PCs

Marcas que reparamos:

Apple

Samsung

Nintendo

Menú de Navegación (Header)

PASO 1: SELECCIONA LA AVERÍA	TU PRESUPUESTO
Dispositivo: Samsung Galaxy S22	Modelo: Galaxy S22
☒ Pantalla Rota 120€	Avería: Pantalla
☐ Batería no carga 45€	
☐ Botones rotos 30€	
☐ Fallo de Software ... 25€	Subtotal: 120.00€
	IVA (21%): 25.20€

	TOTAL: 145.20€

< Volver

Siguiente >

```

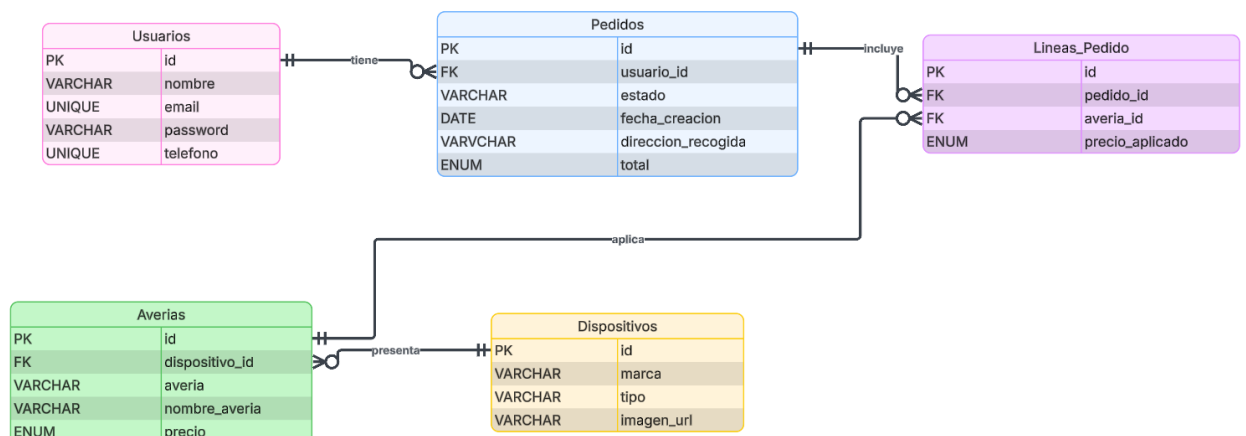
|                                     | [ SOLICITAR REPARACIÓN
] |
+-----+
+-----+-----+
| [ LOGO ADMIN ] | Panel de Gestión > Órdenes |
+-----+-----+
| - Órdenes      | Filtros: [ Todas ] [ Pendientes ] [ En envío
]
| - Clientes      |
| - Precios       | ID      | Cliente | Dispositivo | Estado
|
| - Ajustes       | -----|
|                 | #001   | Juan P. | PS5         | [ Recibido
] |
|                 | #002   | Ana G.  | iPhone 11   | [ Reparado
] |
|                 | #003   | Luis F. | Switch      | [
Pendiente] |
|                 |
| [Cerrar Sesión]| << 1 2 3 >> |
• +-----+
  -+

```

5. Esquema de base de datos

Tablas	Campos	Descripción	Relaciones
Usuarios	Id, nombre, email, password, rol, teléfono	Guarda la información de acceso y contacto. El campo rol nos servirá para mostrar el panel de administrador o no.	1 usuario tiene muchos Pedidos.
Dispositivos	Id, marca, modelo, tipo, imagen_url	Aquí guardaremos los modelos que importemos de la API para no tener que consultarla constantemente.	1 dispositivo tiene muchas Averías.
Averías	Id, dispositivo_id, nombre_averia, precio	Catálogo de precios	1 avería pertenece a 1 Dispositivo.
Pedidos	id, usuario_id, estado, fecha_creacion, dirección_recogida, total	Guarda quién hizo el pedido, cuándo, dónde hay que recogerlo y su estado actual.	1 pedido pertenece a 1 usuario
Lineas_Pedido	Id, pedido_id, averia_id, precio_aplicado	Por si un cliente decide arreglar la pantalla Y la batería en un mismo pedido.	Conecta Pedidos con Averías.

5.2. Modelo entidad relación



6. Configuración del entorno

Mi estructura de proyecto será monorepositorio, es decir, una sola carpeta principal para todo el proyecto, dividida internamente en dos subcarpetas (una para el servidor y otra para la web)

6.1. Preparación de la carpeta y repositorio

Comandos:

```
mkdir proyectoJosetech  
cd proyectoJosetech  
git init
```

6.2. Creación del Backend

Comando para instalar laravel:

```
Composer create-project laravel/laravel backend
```

Esto creará una carpeta llamada `backend` donde irá toda nuestras APIs y la conexión a la base de datos

6.3 Creación del Frontend

Comando para instalar vue:

```
Npm create vue@latest frontend
```

Contestación a las preguntas:

- Project name: frontend
- Add TypeScript?: No
- Add JSX Support?: No
- Add Vue Router for Single Page Application development?: Yes
- Add Pinia for state management?: Yes
- Add Vitest/End-to-End testing?: No
- Add ESLint/Prettier=: Yes

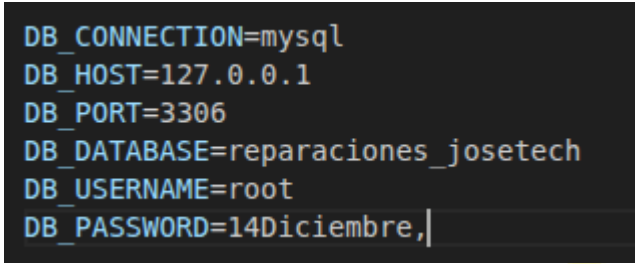
7. Backend

7.1 Creación de la base de datos

Comandos:

```
mysql -u root -p  
CREATE DATABASE reparaciones_josetech;  
QUIT;
```

7.2 Configurar base de datos en el archivo .env



```
DB_CONNECTION=mysql  
DB_HOST=127.0.0.1  
DB_PORT=3306  
DB_DATABASE=reparaciones_josetech  
DB_USERNAME=root  
DB_PASSWORD=14Diciembre,
```

7.3 Crear los modelos y migraciones

Comandos:

```
php artisan make:model Dispositivo -m  
php artisan make:model Averia -m  
php artisan make:model Pedido -m  
php artisan make:model LineaPedido -m
```

8. Configurar los migrations

Dentro de nuestra carpeta de backend, tenemos la siguiente ruta:

/database/migrations

Dentro de la carpeta migrations tenemos todos los archivos .php de las tablas migradas, las configuramos y migramos todo para crear las tablas en la base de datos

```
0001_01_01_000000_create_users_table ..... 245.17ms DONE
0001_01_01_000001_create_cache_table ..... 149.22ms DONE
0001_01_01_000002_create_jobs_table ..... 213.84ms DONE
2026_03_13_182143_create_dispositivos_table ..... 60.98ms DONE
2026_03_13_182156_create_averias_table ..... 289.77ms DONE
2026_03_13_182206_create_pedidos_table ..... 174.00ms DONE
2026_03_13_182223_create_linea_pedidos_table ..... 266.78ms DONE
```

9. Crear las relaciones entre tablas

Dentro de nuestra carpeta de backend, tenemos la siguiente ruta:
/app/Models/User.php

Con sus php de la base de datos, estos archivos tenemos que configurarlos para que Laravel entienda cómo se relacionan entre sí.

10. Creamos el controlador

Para ello, vamos a crear la carpeta Api/ donde guardaremos todo el controlador de la Api

Php artisan make:controller Api/DispositivoController

11. Configuración de Rutas (API)

Al utilizar una arquitectura separada (Frontend en Vue y Backend en Laravel), el servidor debe exponer sus recursos a través de una API REST. A partir de las versiones más recientes de Laravel, el entorno de la API no está habilitado por defecto, por lo que se procedió a su instalación mediante el comando:

php artisan install:api

Esta acción genera el archivo /routes/api.php, el cual actúa como el enrutador principal de nuestra aplicación. En este documento se han definido todos los Endpoints (URLs), separándolos lógicamente en rutas públicas (accesibles por cualquier visitante para consultar el catálogo) y rutas privadas (protegidas por middleware para operaciones sensibles).

12. Población de la Base de Datos

Para poder probar la aplicación en un entorno lo más parecido a la realidad productiva, se ha automatizado la inserción de datos de prueba mediante el uso de Seeders.

En el archivo `/database/seeder/DatabaseSeeder.php` se programó un script que inyecta automáticamente un catálogo masivo de más de 100 dispositivos reales del mercado actual (incluyendo diversas generaciones de Smartphones, Consolas y Ordenadores). Además, el algoritmo asigna aleatoriamente averías comunes a cada dispositivo con tarifas estimadas realistas, y crea un usuario Administrador por defecto.

La inicialización completa de la base de datos se ejecuta mediante el comando:

```
php artisan migrate:fresh --seed
```

13. Controladores de Negocio

Toda la lógica de negocio de la aplicación se ha abstraído en controladores dedicados. Además del controlador de dispositivos, destacan los siguientes:

- **PedidoController:** Es el núcleo transaccional. Se encarga de recibir las nuevas solicitudes de reparación. Como medida de seguridad obligatoria, este controlador ignora los presupuestos enviados desde el cliente (Frontend) y recalcula el coste total consultando directamente la base de datos, evitando así manipulaciones maliciosas. Finalmente, genera un código de seguimiento único alfanumérico para cada orden.
- **AdminController:** Agrupa todas las funciones exclusivas de la gestión del negocio. Valida que el usuario que realiza la petición tenga el rol de 'admin' y permite operaciones como actualizar el estado de las reparaciones o dar de alta nuevas tarifas en el sistema.

14. Seguridad y Autenticación (Laravel Sanctum)

Para la protección de las rutas privadas y la gestión de sesiones, se ha implementado el paquete oficial Laravel Sanctum. Esta herramienta proporciona un sistema de autenticación ligero y robusto basado en Tokens API (Bearer Tokens), ideal para aplicaciones tipo SPA (Single Page Applications) como Vue.js.

Se ha creado un AuthController que gestiona el registro de clientes, el inicio de sesión y el cierre del mismo. Las contraseñas de los usuarios nunca se guardan en texto plano; son encriptadas utilizando el algoritmo Bcrypt. Las rutas críticas del servidor están protegidas mediante el middleware auth:sanctum, el cual rechaza automáticamente cualquier petición HTTP que no incluya un token válido en su cabecera.

15. Configuración de CORS

Dado que el proyecto sigue un patrón cliente-servidor donde el Frontend y el Backend se ejecutan de forma independiente (típicamente en los puertos 5173 y 8000 respectivamente), es obligatorio configurar las políticas de Intercambio de Recursos de Origen Cruzado (CORS).

Para evitar que los navegadores web bloqueen las peticiones por motivos de seguridad, se ha configurado el servidor Laravel modificando el archivo .env de la raíz del proyecto, añadiendo la variable:

```
FRONTEND_URL=http://localhost:5173
```

De esta manera, el servidor autoriza explícitamente a nuestra aplicación web de Vue a consumir los datos de la API sin restricciones.